

Question 2

Problem Statement

- Our team is rewriting the Client Login application and migrating from legacy platform to new platform. As part of this migration, the Front-End UI and back-end Authentication Services are rewritten.
- You need to design the migration architecture to seamlessly migrate from legacy application (UI + Back-end) to the modern application. The migration should not compromise system availability.

What you will deliver

- A migration architecture and design that clearly conveys the candidate's thinking on the challenges above . The candidate decides which artifacts best make their case; You will be assessed for architectural judgment, and distributed systems and integration reasoning, along with the key decisions and trade-offs . The output should be clear, well-reasoned, and presentable to an executive audience — communicating complex architecture crisply to senior, non-specialist stakeholders
-

Scenario: move legacy system consisting of UI, Backend to a modern architecture.

Primary requirement: Users should not have any issues with login/site-access as migration can occur at any point of time

Current system actors : legacy UI, legacy backend , IAM DB and Redis for caching token/auth information

Current login flow assumption:

- Legacy UI routes requests to the legacy backend.
- On every successful login an Id token is generated , that is passed in the response to this authentication request.
- This token is stored in a Redis cache as the key and value is the authentication + authorization(access) details of the logged in user.
- The access details were retrieved from an IAM DB.
- Subsequent requests by the user to the application pass the id token where it is validated with the data in Redis.

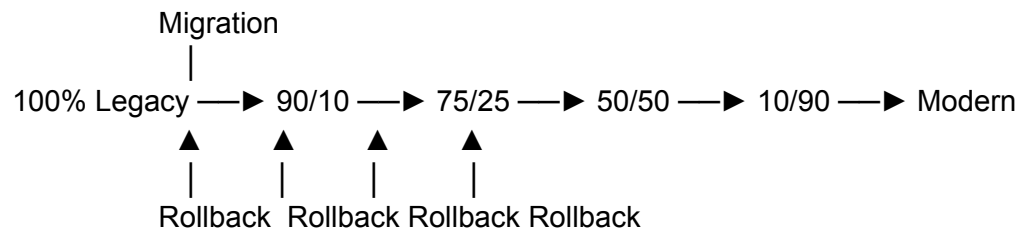
Risk:

- A failure during migration could prevent customers from accessing critical services.

- There is live traffic and the migration should not impact the login / transaction flow

Migration overview:

- Identify core entities
- Run legacy and new platforms in parallel
- Migrate customers incrementally using routing and feature flags
- A customer already authenticated through legacy should be able to transition safely to the modern platform.
- Maintain a quick rollback path for all phases



Phases at a glance:

Analyze/Evaluate -> Prepare -> identify cohort -> migrate cohort-> monitor -> proceed with next cohort/roll back

Migration tools:

- Connection draining (or deregistration delay), - stops the load balancer from sending new requests to the instance while allowing *existing* requests to finish.
- Feature flags - tools like Launch darkly can be used to tag the individuals of a cohort

Assumption:

The same DB is used by legacy and the new applications and no DB migration is needed.

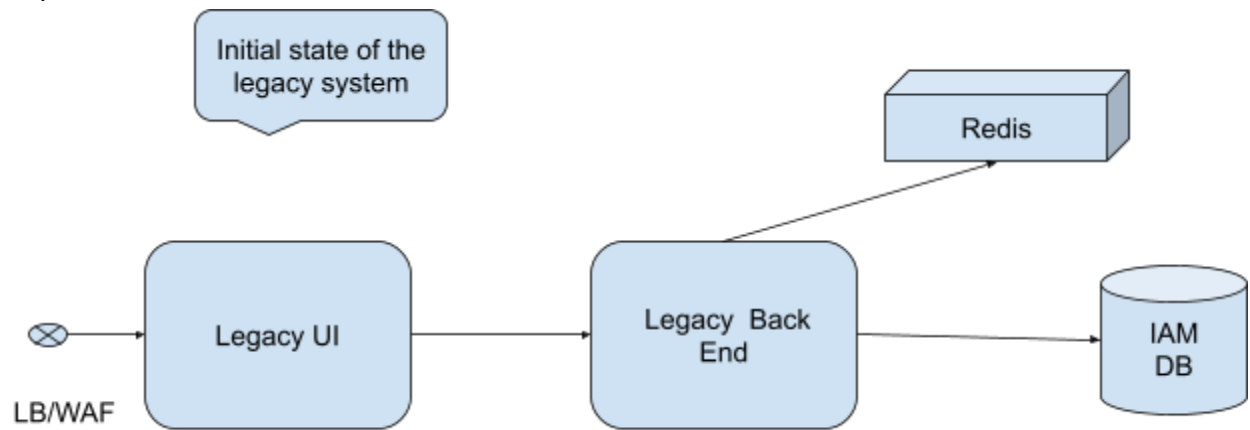
Migration steps -

- These steps are initially tried and tested on lower environments and all possible risk and mitigation identified before doing it on production.
- The approach chosen here is a widely proven strategy in which the source(old) platform is enhanced to a level such that it helps with the smooth transition.
- The overall risk is less with 0% downtime.

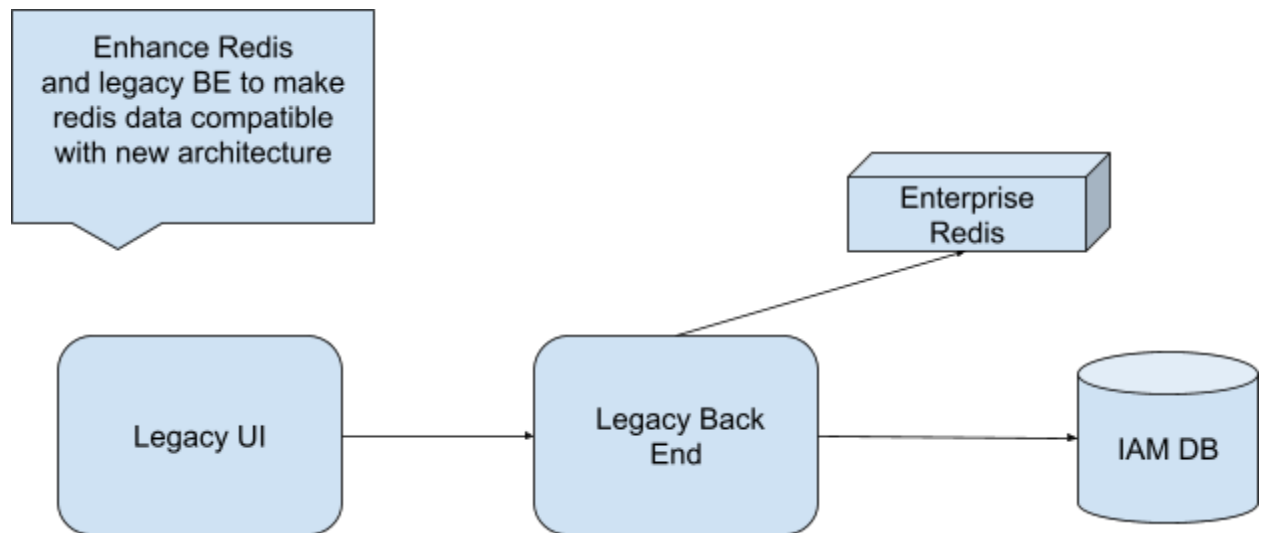
Step	Overview	Details	AI involvement	Effort/Risk
1	Prep the legacy UI and Backend layers for migration	Enable to route traffic based on userid feature flags. The flags can be based on individual users or at a cohort level. Enablement can be done by integrating with tools like launch darkly Tools that route based on userID: • AWS WAF + Application Load Balancer (ALB) • Cloudflare • NGINX / NGINX Plus WAF Enrich the redis values such that it is compatible with both the legacy and new backend's needs. Legacy backend might find some of the new fields redundant. Functionality will not be impacted Connection drain strategy can be used if required to install the new changes Load balancers are used between the layers	AI can be used to identify compatibility that would help with the migration.	Medium
2	Prep the cache if required (The target is a distributed cache)	Upgrade of Redis a new version of redis if needed. (like a distributed enterprise redis). Connection drain strategy should be used if required	Use AI to monitor any abnormalities in redis behavior.	Low

3	Monitor	Observe for system stability	AI can be used to proactively catch degradation quickly for example	Low
4	Introduce new FE and BE components	These components will be deployed and run in parallel with no traffic and will have the modern architecture. The new FE and BE will make token related objects that are compatible with the legacy system	New components are developed using AI	Low
5	Identify the cohort	Could be test users/ internal users/ users identified as less impactful or a small cohort	AI can be used to identify the cohort that will be less impacted (like identifying users with least logins over a period of time)	Low
6	Implement the change & monitor the cohort	Migration feature flags are turned on for the identified cohort. Start with a low percentage in the low single digits The legacy system is flexible to route to the new platform		Low
7	More Cohort are migrated with the confidence gained from the metrics/stability			Low

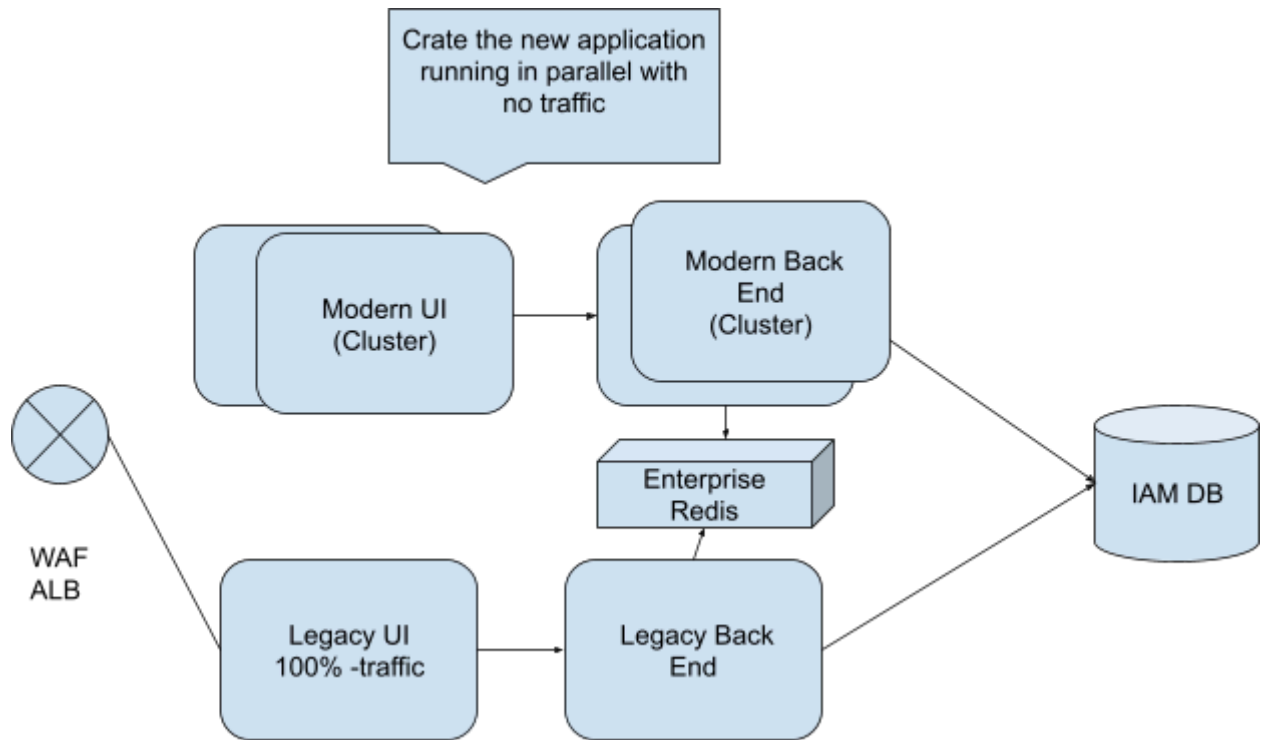
Step 0



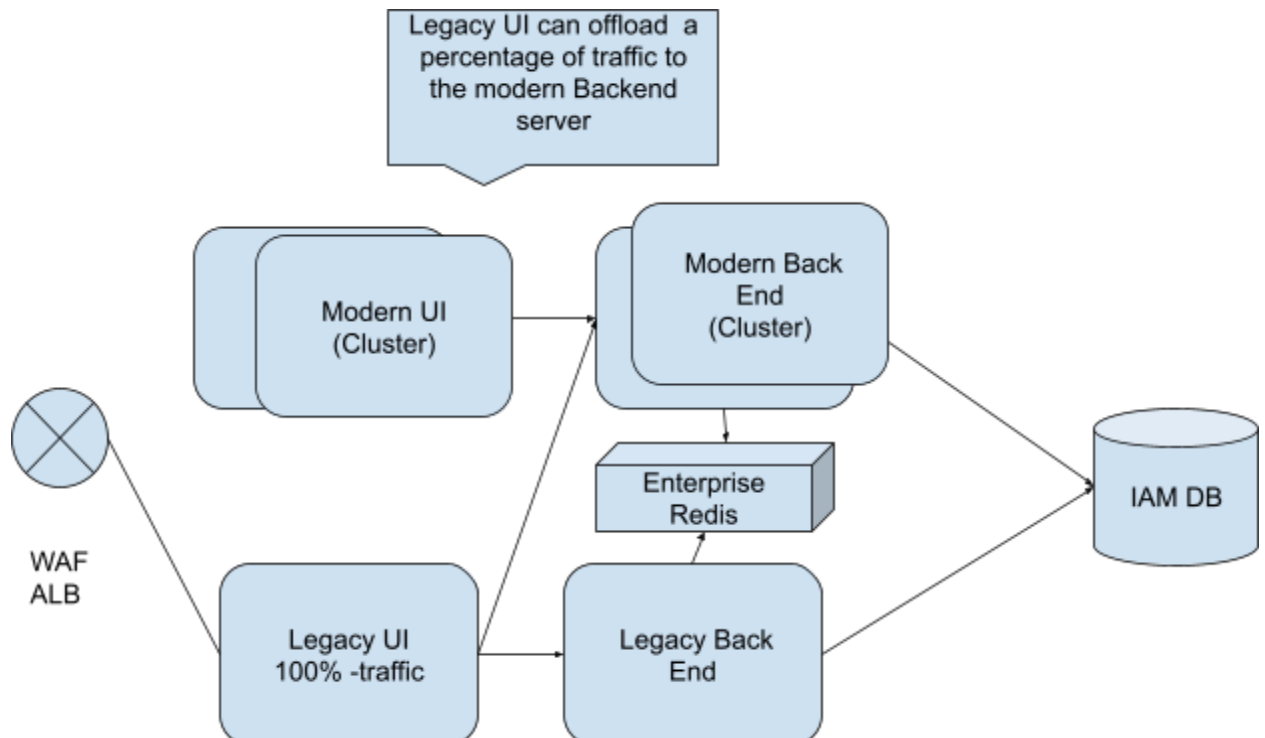
Step 1



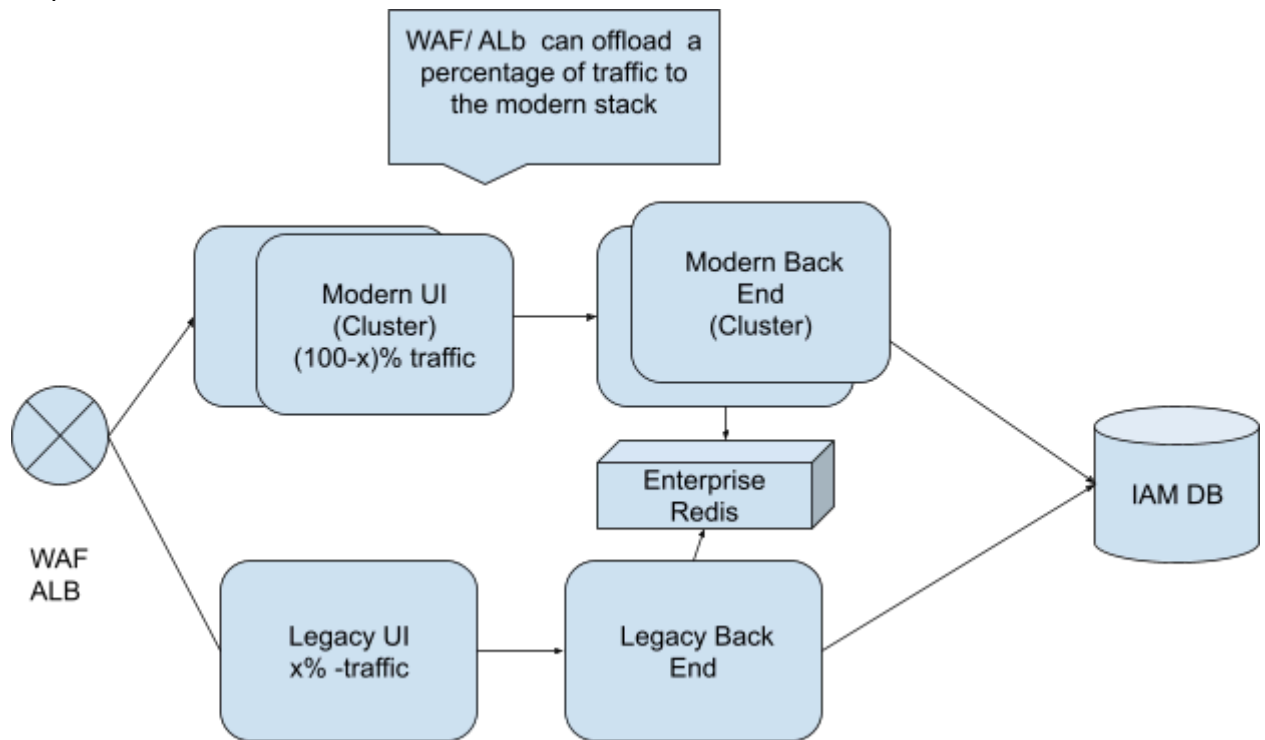
Step 2



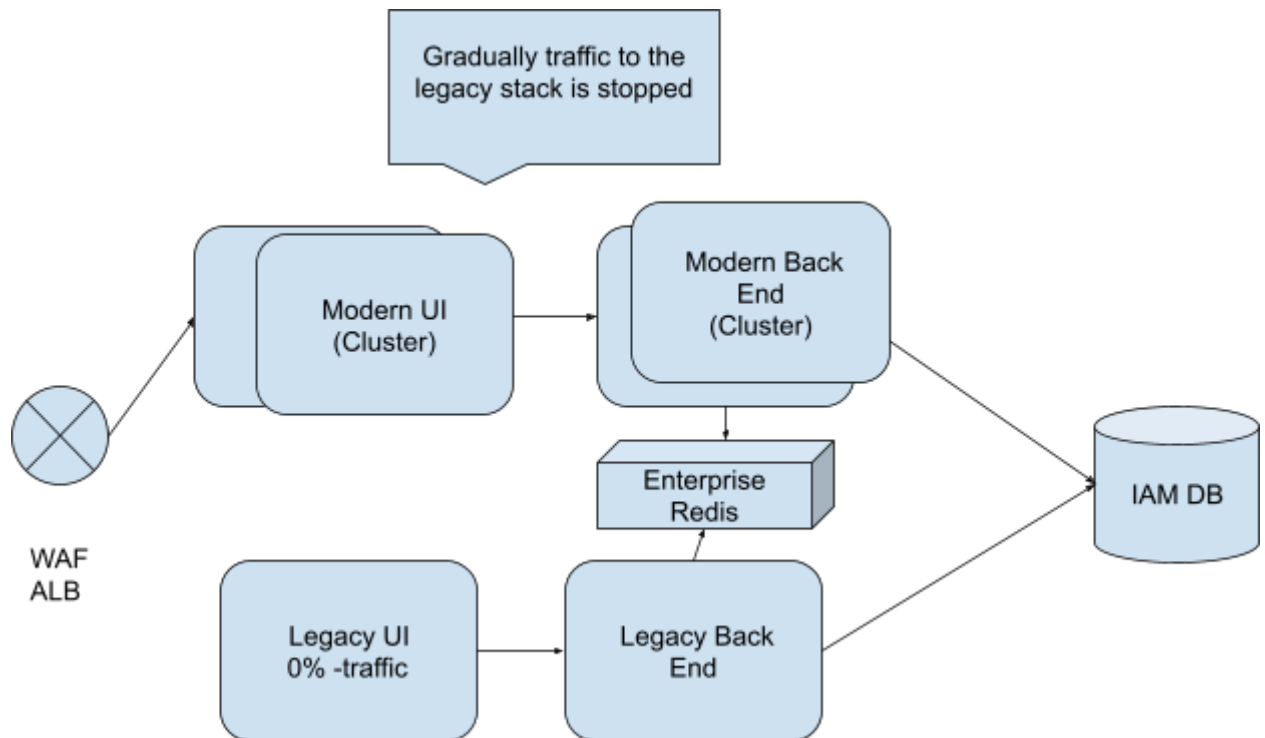
Step3



Step4



Step5



Notes:

Sample cohort identification:

Modern Login

— Employee cohort	ON
— Pilot cohort	OFF
— Region A	OFF
— Region B	OFF
— General population	5%

Why I Would Not Do a Big-Bang Cutover:

A big-bang approach would look like:

Legacy



Modern

The problem is that we don't know whether the modern application will behave correctly under:

- Peak traffic
- Unexpected authentication patterns
- Different browsers/devices
- Existing customer sessions
- Password/account edge cases
- MFA flows
- Fraud/security controls
- Downstream dependencies
- Regional failures

A problem discovered after 100% cutover becomes a production incident.

Observability

I would build migration observability before increasing traffic.

A single dashboard should show:

Customer Metrics	Technical Metrics	Security Metrics	Migration Metrics
• Login success rate • Login failure rate • MFA success rate • Password reset success • Session establishment • Session expiration • Customer abandonment	• Latency • Error rate • CPU/memory • Dependency failures • Database latency • Authentication service latency	• Authentication anomalies • Suspicious login patterns • Token failures • Account lockouts • Fraud/security signals	• Legacy traffic % • Modern traffic % • Cohort distribution • Rollback events • Migration completion %

Go/No-Go Criteria

I would never say:

"The new platform looks good. Let's move everyone."

Instead, every migration stage would require measurable acceptance criteria.

For example:

1. Availability: No material degradation in login availability
2. Performance: p95/p99 latency within agreed thresholds
3. Security: No critical security findings, No increase in authentication anomalies
4. Reliability: Error rate within baseline
5. Customer Experience: Login completion rate equal to or better than legacy
6. Operations: Monitoring and alerting validated

Rollback

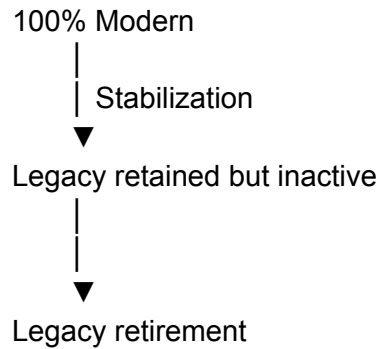
Rollback tested successfully . Only when all criteria are satisfied would we increase traffic.

Legacy Retirement

One mistake I would avoid is declaring victory when 100% of traffic moves to modern.

I would establish a stabilization period.

For example:



During stabilization, we would confirm:

- No unexpected traffic
- No critical dependencies
- No customer cohorts missed
- No reconciliation issues
- No operational gaps

Only then would I decommission legacy.

Risk Model

I would summarize the migration risks for executives this way:

Risk	Mitigation
Modern application failure	Progressive rollout
Authentication/session incompatibility	Common identity/session contract
Data inconsistency	Single source of truth + reconciliation
Security regression	Automated security controls + security gates
Performance degradation	Canary + real-time monitoring
Regional failure	Multi-region architecture

Customer impact

Cohort-based rollout

Slow rollback

Automated traffic control

Unknown legacy dependency

Parallel operation + dependency mapping

Operational readiness

Runbooks + game days

Fallback and Blast-Radius Control

- **Automated Active-Passive Circuit Breakers** configured at the API Gateway. If the Modern Platform exhibits an error rate $>1\%$ over a 10-second window, the Gateway instantly flips the traffic dial back to 0%, routing all new login attempts to the proven Legacy Platform